Lesson Outcomes

At the end of this chapter, student should be able to:

- ✓ Describe what a computer program is
- ✓ Explain the importance of programming to computer use
- ✓ Appreciate the importance of good programs
- ✓ Explain the relationship between compilers, interpreters and programs
- ✓ Recognize program errors
- ✓ Become familiar with the program design process

References:
Malik, D.S., (2010). *C++ Programming, From Problem Analysis to Program Design*. Course Technology, Canada.
Gary, R.B., (2010). *C++ For Engineers & Scientists: 3rd Edition.* Course Technology, Canada.

A Brief History of C++ Language

- C++ (OOP) is built upon the foundation of C (structured –*imperative*- programming language).
- C++ is a superset of C and C++ compilers can be used to compile C programs.
- C++ is an expanded and enhanced version of C that embodies the philosophy of object-oriented programming.
- C was invented and first implemented by Dennis Ritchie in 1970s, while C++ invented by Bjarne Stroustrup in 1979 at Bell Laboratories in Murray Hill.
- The reason for invention of C++ is to cater the increases of complexity in computer problem solving or programming.

Introduction to Programming

A. What is a computer program

1. Program

A program is a set of instructions that direct the computer to accomplish specific tasks. Another term commonly used for computer programs is *software***,** which describes all programs that are used in particular installation together with the associated documentation.

2. Programming languages and its types.

Programming is also known as software engineering. It is the process of writing a program, or software. The instructions that can be used to construct a program consist of statements written in a ***programming language***, the language that the computer can interpret or "understand" such as C++ or Visual BASIC. First introduced in 1945, programming languages have evolved over the years.

The births of the generations are as follow.

- First generation, 1945 - machine language
- Second generation, mid-1950s – assembly language
- Third generation, early 1960s – high-level languages: FORTRAN, COBOL, C, BASIC, C++, and Pascal

References:
Malik, D.S., (2010). *C++ Programming, From Problem Analysis to Program Design*. Course Technology, Canada.
Gary, R.B., (2010). *C++ For Engineers & Scientists: 3rd Edition.* Course Technology, Canada.

a. First Generation: Machine Language (Low-Level Language)

- The only programs that can be used to operate a computer.
- The lowest level of computer programming language.
- Machine language or also referred to as *executable programs*, or executables for short, consist of a sequence of instructions composed of binary numbers (0's and 1's) such as:

```
10000000  00000001  00000010
11110000  00000010  00000011
```

  which correspond to *on* and *off* of electrical states of the computer, which not convenient for people to read and use.
- Each instruction in a machine language consists of two parts: **instruction** part and **address** part. The instruction part is referred to as opcode (operation code), is usually the leftmost set of bits in the instruction. It tells the computer operation to be performed, such as add, subtract, multiply, etc., while the rightmost bits specify the memory addresses of the data to be used.
- For example, assuming that the eight leftmost bits of the first instruction (i.e. 100000000) listed above is the opcode for add, and the next two groups of twelve bits are the addresses of two operands to be added. This instruction would be a command to "add the data in memory location 1 to the data in memory location 2."
- Similarly, assuming that the opcode 11110000 means multiply, the next instruction is a command to "multiply the data in memory location 3 by the data in location 4".

b. Second Generation: Assembly Language (Low-Level Language)

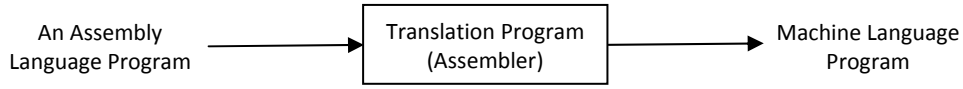
- One of the first advances in programming was the substitution of word-like symbols, such as ADD, MUL, for the binary opcodes and both decimal numbers and label for memory addresses.
- For example, the previous two machine language instruction can now be written as

```
ADD 1, 2
MUL 2, 3
```

- The particular programming language is called **assembly language**. Assembly language is a low-level language that allows a programmer to use abbreviations or easily remembered words instead of numbers.

References:
Malik, D.S., (2010). *C++ Programming, From Problem Analysis to Program Design*. Course Technology, Canada.
Gary, R.B., (2010). *C++ For Engineers & Scientists: 3rd Edition.* Course Technology, Canada.

- Computer can only execute machine language programs, the set of instructions contained within an assembly language program must be translated into machine language. Translator programs that perform this function for assembly language programs are known as assemblers.



- Both machine and assembly languages are classified as low-level languages, and they generally execute at the fastest level possible. As with the case of machine language, assembly language varies from computer to computer – it is machine dependent.

c. Third Generation: High-level language

- A high-level language is an English-like programming language. A high-level language program can be run on a variety of computer types.
- For examples: FORTRAN, Pascal, Java, C++, C, COBOL, ADA, LISP, PL/1, RPG, Logo, APL, FORTH and Prolog.
- As with assembly language, high-level language requires a translator to translate it into machine language. The translator for high-level languages is either a compiler or an interpreter (compiler and interpreter will be discussed later in this topic).

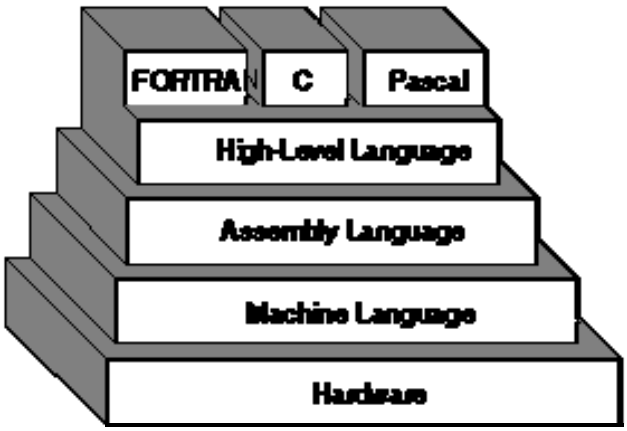


Figure 1: Hierarchy of Computer Organization

3. Programming, programmers and users

The process of writing or coding programs is called *programming*, and the individuals who write programs are called *programmers*. *Users*, on the other hand, are individuals who use the program.

References:
Malik, D.S., (2010). *C++ Programming, From Problem Analysis to Program Design*. Course Technology, Canada.
Gary, R.B., (2010). *C++ For Engineers & Scientists: 3rd Edition.* Course Technology, Canada.

B. Importance of computer programming

- A computer without a program is useless. A computer must have a program to accept input, process the input and produce output.
- Before a program is written, a programmer analyzes the requirement and designs a correct algorithm so that the program will produce the desired output. Therefore, it is very important that programmers have a good understanding of computer, problem-solving approaches and compositions of the chosen programming languages.

C. Importance of good programs

In order for a program to be considered as a good program, it must have the following:

1. Reliability of output

A good program must be able to produce correct output. For that, a different set of input data is used to ensure the reliability of the output.

2. Program's efficiency

A good program must be designed to be efficient and reliable in the sense that it produces no errors during execution process. Also, the program must achieve its purpose so that the final result can be trusted. Thus, it is important that the program is outlined first using the pseudocode or flowchart tool.

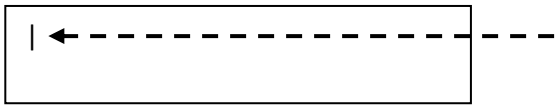
3. Interactivity

The interaction process between the user and the program must be well defined. The interactivity is important so that the user knows the processing status. Program that are user-friendly allow the users to responds to instruction correctly and this will help the users to key in input thus minimizing the errors resulted from invalid data.

References:
Malik, D.S., (2010). *C++ Programming, From Problem Analysis to Program Design*. Course Technology, Canada.
Gary, R.B., (2010). *C++ For Engineers & Scientists: 3rd Edition.* Course Technology, Canada.

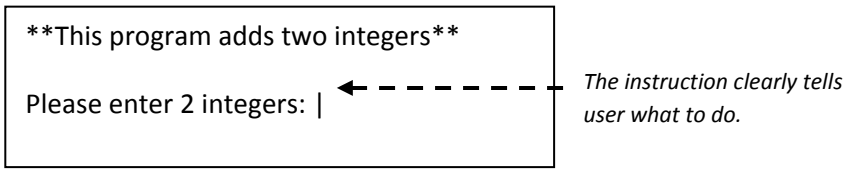
Example of a bad program's output screen

This output produces the blinking cursor that expects the user to enter some data. However, the kind of data to be entered is unknown to the user.



Example of a good program's output screen

This output screen tells the user what will be achieved when entering the input data. The instruction helps the user to enter data to be correctly processed by the program.



4. Program readability

Readability is concerned with how other person views one's program. For programmers, the use of indention and comments are common to improve the program's readability.

a. Indentation

- Indentation helps in making the structure of the program clearer and easier to read. A statement within a statement should be indented to show the user which statements are subordinated of the other.
- In C++, particularly, if-else, while and do-while statements should be indented. Embedded braces { } are also indented to make it easier to find the matching pairs.

References:
Malik, D.S., (2010). *C++ Programming, From Problem Analysis to Program Design*. Course Technology, Canada.
Gary, R.B., (2010). *C++ For Engineers & Scientists: 3rd Edition.* Course Technology, Canada.

Table1: Example of indentation using if-else, for and while statement.

Before indentation:	**After indentation:**

`if (score > 90)`
`{ highest = highest + 1;`
`cout << "Highest scores!!"; }`
 	
`else if ( score < 40 )`
`{ lowest = lowest + 1;`
`cout << "Lowest scores!!";}`
 	
`for ( int bil = 1; bil < 5; bil++)`
`{ sumAllNum = sumAllNum + num;`
`cout << "\n Enter num : ";`
`cin >> num; }`
 	
`while (y >= 2)`
`{ x = x / 2;`
`if (x > 0 )`
`y = 1 + y;`
`else`
`y = x + y;`
`i++;}`
	`if (score > 90)`
	`{ highest = highest + 1;`
	`cout << "Highest scores!!";`
	`}`
	`else if ( score < 40 )`
	`{ lowest = lowest + 1;`
	`cout << "Lowest scores!!";`
	`}`
	
	`for ( int bil = 1; bil < 5; bil++)`
	`{ sumAllNum = sumAllNum + num;`
	`cout << "\n Enter num : ";`
	`cin >> num;`
	`}`
	
	`while (y >= 2)`
	`{ x = x / 2;`
	`if (x > 0 )`
	`y = 1 + y;`
	`else`
	`y = x + y;`
	`i++;`
	`}`

b. Comments

- Some explanatory notes or comments (sometimes referred to as internal documentation) should be place in the program coding to improve its readability. In other words, comments are there for the convenience of anyone reading the program.
- Comments can be placed anywhere within in a program and they will **NOT** be executed by the compiler. Commenting out a section of code is very useful in a debugging process.

References:
Malik, D.S., (2010). *C++ Programming, From Problem Analysis to Program Design*. Course Technology, Canada.
Gary, R.B., (2010). *C++ For Engineers & Scientists: 3rd Edition.* Course Technology, Canada.

- C++ supports two types of comments:

  i. Line comment

  A line comment begins with two slashes (//) and continues to the end of line.

Example 1:

```
//This program is to calculate the perimeter – this is line //comment
#include <iostream.h>

int main()
{
	//declare variables – this is line comment
	float perimeter = 0.0, length = 10.5, width = 8.6;

	//calculate perimeter
	perimeter = (length + width) * 2;

	//print to the screen console
	cout << "Perimeter is " << perimeter;
return 0; //indicates program ended normally – this is comment
} //end of main function – this is line function
```

Example 2:

```
//This program is to calculate area of the circle – this is line //comment
#include <iostream.h>

int main()
{	//declare variables – this is line comment
	float radius = 5.6, area;
	float const PI = 3.142;

	//calculate area of the circle – this is line comment
	area = PI * radius * radius;

	//print to screen console
	cout << "The area of the circle is " << area;
return 0; //indicates program ended normally – this is comment
} //end of main function – this is line function
```


References:
Malik, D.S., (2010). *C++ Programming, From Problem Analysis to Program Design*. Course Technology, Canada.
Gary, R.B., (2010). *C++ For Engineers & Scientists: 3rd Edition.* Course Technology, Canada.

ii. Block comment

Block comment begins with the symbol `/*` and end with symbol `*/`. Block comments are conveniently used for statements that span across two or more lines.

Example:

```
/*
Programmer  : Roslan Sadjirin
Course Code : CSC125 / CSC128
This program is to calculate the perimeter – this is block comment
*/

#include <iostream.h>

int main()
{
	//declare variables – this is line comment
	float perimeter = 0.0;
	float length = 10.5;
	float width = 8.6;

	//calculate perimeter
	perimeter = (length + width) * 2;

	//print to screen console
	cout << "Perimeter is " << perimeter;

return 0; /*indicates program ended normally – this is block
		comment*/

} //end of main function – this is line function
```


References:
Malik, D.S., (2010). *C++ Programming, From Problem Analysis to Program Design*. Course Technology, Canada.
Gary, R.B., (2010). *C++ For Engineers & Scientists: 3rd Edition.* Course Technology, Canada.

D. Relationship between compilers, interpreters, assemblers and programs

1. What are the compiler and interpreter?

- Compiler and interpreter is a **language translator program** that translates source code (human-readable code) into machine language.

Language Translator Program

Compiler

- Translate all of the instructions of high-level language as a complete unit.
- The original program that is written in high-level language is called **source code**.
  - Source code - the original program written in assembly language or high-level languages. The file containing the source code is called source file, and in C++, it is uses the filename extension ***.cpp*** (*cpp stands for C plus plus*)
- While attempting to translate each of the statement, the compiler also detects error message after translation is completed.
- Otherwise, the compiler successfully translates the source code into machine language, which in this case **object code**.
  - Object code – created from the compilation process in machine readable form. The file containing the object code is called the object file, and it uses the file name extension ***.obj*** (*obj stands for object*)
- After compiler creates the object file, it then invokes another program called a **linker**. The linker combines the object file with other machine code necessary for the C++ program to run correctly. The linker files then produces an executable file that has an extension of ***.exe*** (*exe stands for executable*).

Interpreter

- Converts high-level language instruction into machine language one at a time, in succession.
- Any of the instructions that contain error, interpreters provides immediate feedback.
- Does not produce object code.

References:
Malik, D.S., (2010). *C++ Programming, From Problem Analysis to Program Design*. Course Technology, Canada.
Gary, R.B., (2010). *C++ For Engineers & Scientists: 3rd Edition.* Course Technology, Canada.

2. *Steps in running a program using compiler*

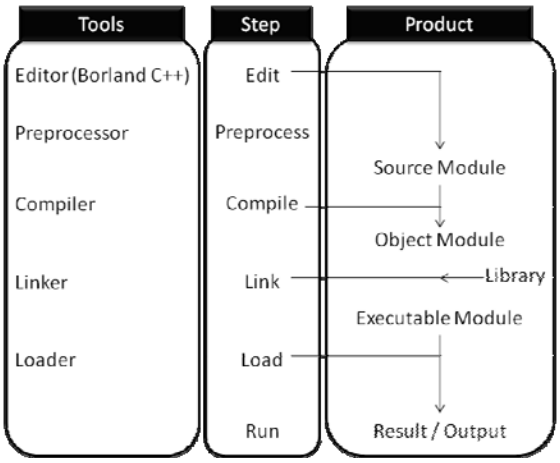


Figure 2: Steps in running a program using compiler

3. *Steps in running a program using interpreter*

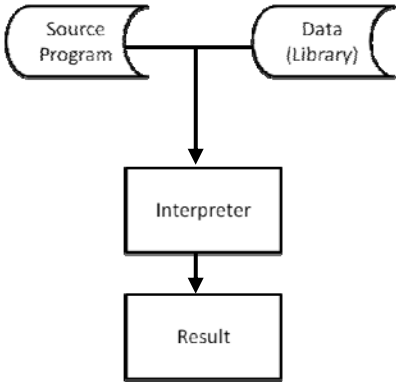


Figure 3: Steps in running a program using interpreter

4. *Program errors*

Although the principle in programming is for efficiently produce readable and error-free programs that work correctly, *errors* or sometimes are called *bugs* can occur at any time.

There are different types of errors:

a. Run-time errors

b. Syntax errors

c. Logic errors

References:
Malik, D.S., (2010). *C++ Programming, From Problem Analysis to Program Design*. Course Technology, Canada.
Gary, R.B., (2010). *C++ For Engineers & Scientists: 3rd Edition.* Course Technology, Canada.

a. Run-time errors

Run-time errors occur during the execution of a program and are detected by the compiler.

Example:

```
1   #include <iostream.h>
2
3   int main()
4   {
5         int a = 0 , b = 7, result; // declaration
6         result = b / a;
7
8         cout << "The result is " << result << endl; /*Run-time Error
9                                                       here!*/
10
11  return 0;
12  }
13
```

Attempting to divide by zero in the above statement causes a run-time error such as "Floating point division by error" or divide-error exception". The method for detecting errors after a program has been executed is called debugging process. A debugger program is available in C++ for detecting errors while a program is being executed.

b. Syntax errors

A syntax error is an error in the structure or spelling of a statement. This error can be detected by the compiler during compilation of the program.

Example:

```
1   #include <iostream.h>
2
3
4   int main()
5   {
6         cout << "/nThere are 5 syntax errors here"
7         cot >> "correct them;
8
9   return 0;
10  }
```


References:
Malik, D.S., (2010). *C++ Programming, From Problem Analysis to Program Design*. Course Technology, Canada.
Gary, R.B., (2010). *C++ For Engineers & Scientists: 3rd Edition.* Course Technology, Canada.

Explanation:

i. Invalid use of backslash (/) in line 6
ii. Missing semicolon (;) in line 6
iii. Keyword `cout` is misspelled in line 7
iv. Invalid use of insertion symbol (>>) in line 7
v. Missing a closing quote (“) in line 7

A C++ statement with correct syntax will be accepted by the compiler without any error messages. Nevertheless, a statement or program can be syntactically correct but still be logically incorrect. Consequently, incorrect result would be produced.

c. Logic errors

Logic errors refer to the unexpected or unintentional errors resulted from some flaw in the program's logic. Logic error is very difficult to detect since compiler cannot detect this error. The only way to detect it is by testing the program thoroughly and comparing its output against manually calculated results.

Some indications of logic errors include:

i. No numerical output

This is caused by a missing numerical output in a `cout` statement.

Example:

```
1   #include <iostream.h>
2
3   int main()
4   {
5         int qty;
6         double price;
7         cout << “Enter quantity of items purchased: ”;
8         cin >> qty;
9         cout << “\n Enter the price unit: RM ”;
10        cin >> price;
11        cout << “The total price is RM ”;    // logic error here
12        cout << “\n Thank you.”;
13  return 0;
14  }
```


References:
Malik, D.S., (2010). *C++ Programming, From Problem Analysis to Program Design*. Course Technology, Canada.
Gary, R.B., (2010). *C++ For Engineers & Scientists: 3rd Edition.* Course Technology, Canada.

The output for the above program statement is

```
Enter quantity of items purchased: 5
Enter the price unit: RM 50;
The total price is
Thank you.
```

ii. Unappealing or misaligned output

This either caused by an error in `cout` statement or the use of formatting output statement (`\n, \t`).

Example:

```
1   #include <iostream.h>
2
3   int main()
4   {
5         int qty;
6         double price, total;
7         cout << "Enter quantity : \t\t "; // logic error here
8         cin >> qty;
9         cout << "\n Enter price: RM ";
10        cin >> price;
11        total = qty * price;
12        cout << "The total price is RM " << total;
13        cout << "Thank you."; // logic error here
14  return 0;
15  }
```

Sample output

```
Enter quantity:           5
Enter price: RM 10
The total price is RM 50Thank you
```


References:
Malik, D.S., (2010). *C++ Programming, From Problem Analysis to Program Design*. Course Technology, Canada.
Gary, R.B., (2010). *C++ For Engineers & Scientists: 3rd Edition.* Course Technology, Canada.

iii. Incorrect numerical results

This is caused by either incorrect values or data types assigned to the variables used in an expression, incorrect arithmetic expression, an omission of a statement, round off error, or improper use sequence of statement.

Example:

```
int main()
{
	char fnum = 10;		// error here
	int snum = 5;
	int sum = fnum + snum;
	.
	.
	return 0;
}
```

Where `fnum` is of type character. The valid character to be assigned is

```
char fnum = 'A';
```

iv. Premature program termination

if the error is detected during the program execution, a run-time error occurs that results in an error message being generated or abnormal and premature program termination.

Example:

```
1  int radius = 4.562, num = -4, count = 0;
2  float total = 10.0, average;
3  average = total / count;
4  cout << "Average is ";
5  cout << "\n The square root of " << num << " is " << sqrt(num);
```

Explanation:

The logic errors are:

i. Assigning invalid values to variable of type integer in line 1

ii. Attempting to divide by zero in line 3

iii. Missing numerical output in line 4

iv. Taking the square root of a negative number in line 5

References:
Malik, D.S., (2010). *C++ Programming, From Problem Analysis to Program Design*. Course Technology, Canada.
Gary, R.B., (2010). *C++ For Engineers & Scientists: 3rd Edition.* Course Technology, Canada.

E. Program Development Life Cycle

- In designing a program, there is no complete set of rules or specific algorithm to follow. However, software developers try to use a reasonably consistent problem solving approach for constructing computer programs.
- The phases in the problem solving approach are outlined as follows:

  Phase 1: Problem Definition (Analysis)

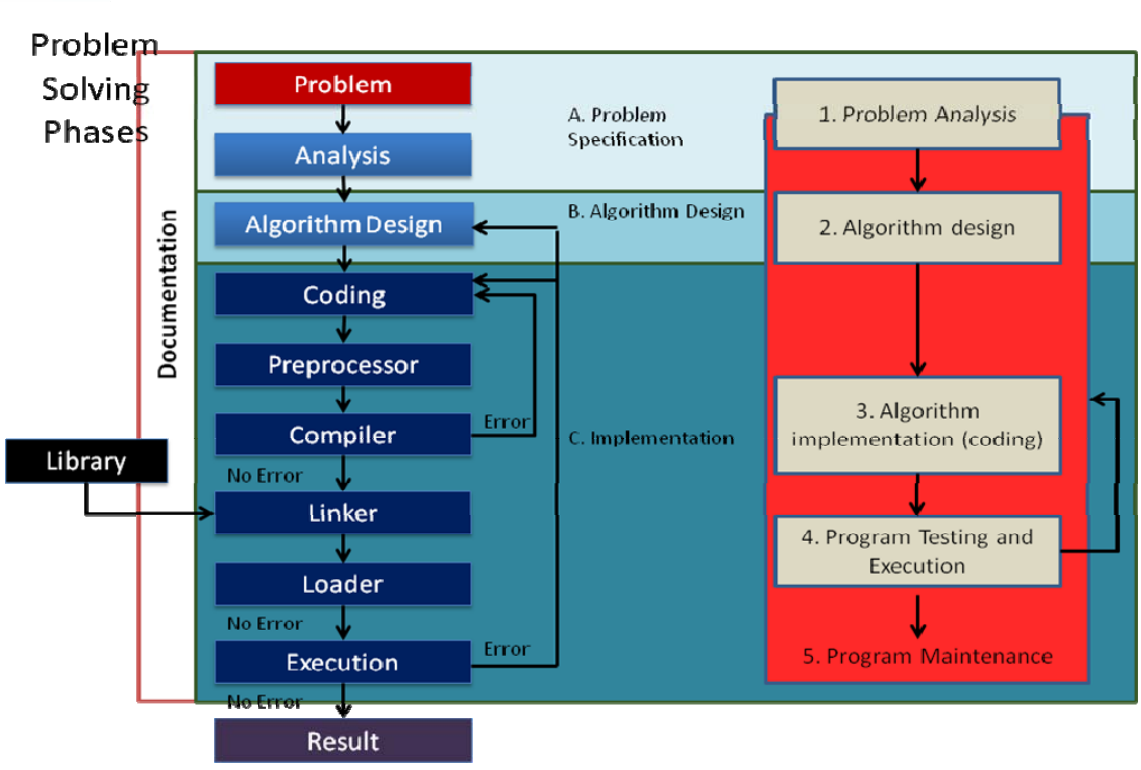
  Phase 2: Algorithm design

  Phase 3: Algorithm implementation

  Phase 4: Program testing

  Phase 5: Program maintenance

- The phases can be further divided into two:
  a. Problem solving phases: consist of problem definition and algorithm design
  b. Implementation phases: consist of algorithm implementation, program testing and program maintenance phases.



References:
Malik, D.S., (2010). *C++ Programming, From Problem Analysis to Program Design*. Course Technology, Canada.
Gary, R.B., (2010). *C++ For Engineers & Scientists: 3rd Edition.* Course Technology, Canada.

1. Problem definition (Analysis)

Problem definition is also called the analysis phase. The problem is defined to obtain a clear understanding of the problem requirement.

The following questions should be asked to get a complete problem specification:

a. What are the input data?
b. What are the output (desired) data?
c. What formula is to be used?
d. What other assumptions or constraints can be made?
e. What is the expected output screen?

2. Algorithm design

The specifications derived earlier in the analysis phase are translated into the algorithm. An algorithm is a step-by-step sequence of precise instructions that must terminate and describes how the data is to be processed to produce the desired outputs. The instruction may be expressed in a human language.

An algorithm must satisfy some requirements:

a. Input and output

It must have zero or more input and must produce at least one output.

b. Unambiguous

Every step in algorithm must be clear as to what it is supposed to do and how many times it is expected to be executed.

c. Correct and efficient

It must be correct and efficiently solve the problem for it is designed.

d. Finite

It must execute its instruction and terminate in a finite time.

An algorithm can be written or described using several tools:

a. Pseudocode

Use English-like phrases to describe the processing task. It is not standardized since every programmer has his or her own way of planning the algorithm.

References:
Malik, D.S., (2010). *C++ Programming, From Problem Analysis to Program Design*. Course Technology, Canada.
Gary, R.B., (2010). *C++ For Engineers & Scientists: 3rd Edition.* Course Technology, Canada.

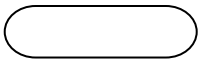
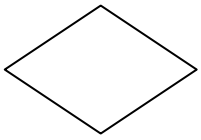
Table 2: Pseudocode style

Style 1	Style 2 (Modular design)
Problem Begin 1. Subproblem 1 Task 1,1 Action 1,1,1 Action 1,1,2 2. Subproblem 2 Task 1,2 Action 1,2,1 Action 1,2,2 End	Problem Begin 1. Subproblem 1 2. Subproblem 2 3. End 1. Subproblem 1 Task 1,1 Action 1,1,1 Action 1,1,2 2. Subproblem 2 Task 1,2 Action 1,2,1 Action 1,2,2 End

b. Flowchart

Use standardized symbol to show the steps the computer needs to take to accomplish the program's objective. Because flowcharts are cumbersome to revise, they have fallen out of favour by professional programmers. Pseudocode, on the other hand, has gained increasing acceptance.

Table 3: Flowchart symbols

Name	Symbol	Description
Lozenge		Terminal (start / stop)
Parallelogram		Data (input / output)
Rectangle		Process
Diamond		Decision / Selection
Rectangle		Module (Pre-defined process)
Arrow		Flow line (flowchart)

References:
Malik, D.S., (2010). *C++ Programming, From Problem Analysis to Program Design*. Course Technology, Canada.
Gary, R.B., (2010). *C++ For Engineers & Scientists: 3rd Edition.* Course Technology, Canada.

Most computer algorithms consist of at least the following processes.

i. Get the input (data)

ii. Perform the computation (processing)

iii. Display the output (results)

3. Algorithm implementation

The algorithm is translated into a computer program by using a specific programming language, for example C++. The process called coding, which involves editing, compiling and debugging.

4. Program testing

Program testing requires testing the completed program to verify that it produces expected output. A different set of testing data is normally used to verify that the program works properly and that it is indeed solving the given problem.

5. Program maintenance

Often, there may be new requirements to be added into the current program. Making revisions to meet the changing needs with ongoing correction of problem are the major efforts in the program maintenance. As a result, the program codes may be modified, added or deleted accordingly. Thus, it is very important that a program is well documented for future development.

References:
Malik, D.S., (2010). *C++ Programming, From Problem Analysis to Program Design*. Course Technology, Canada.
Gary, R.B., (2010). *C++ For Engineers & Scientists: 3rd Edition.* Course Technology, Canada.

F. Example of problems that can be solved using computers

Problem 1: Write a complete C++ program to calculate the sum and average of 2 integer numbers, and then display the sum and the average of those numbers.

Step 1: Problem definition (analysis)

Output	Input	Process
Output variable: sum average	**Input variable:** number_1 number_2 **Constant:** ***NONE***	**Processing items:** sum, average **Formula:** sum = number_1 + number_2 average = sum / 2 **Step / Solution algorithm:** 1. Get input (*from keyboard*) 2. Calculate sum (*in CPU and Memory*) 3. Calculate average (*in CPU and Memory*) 4. Display output (*to screen console / monitor*)

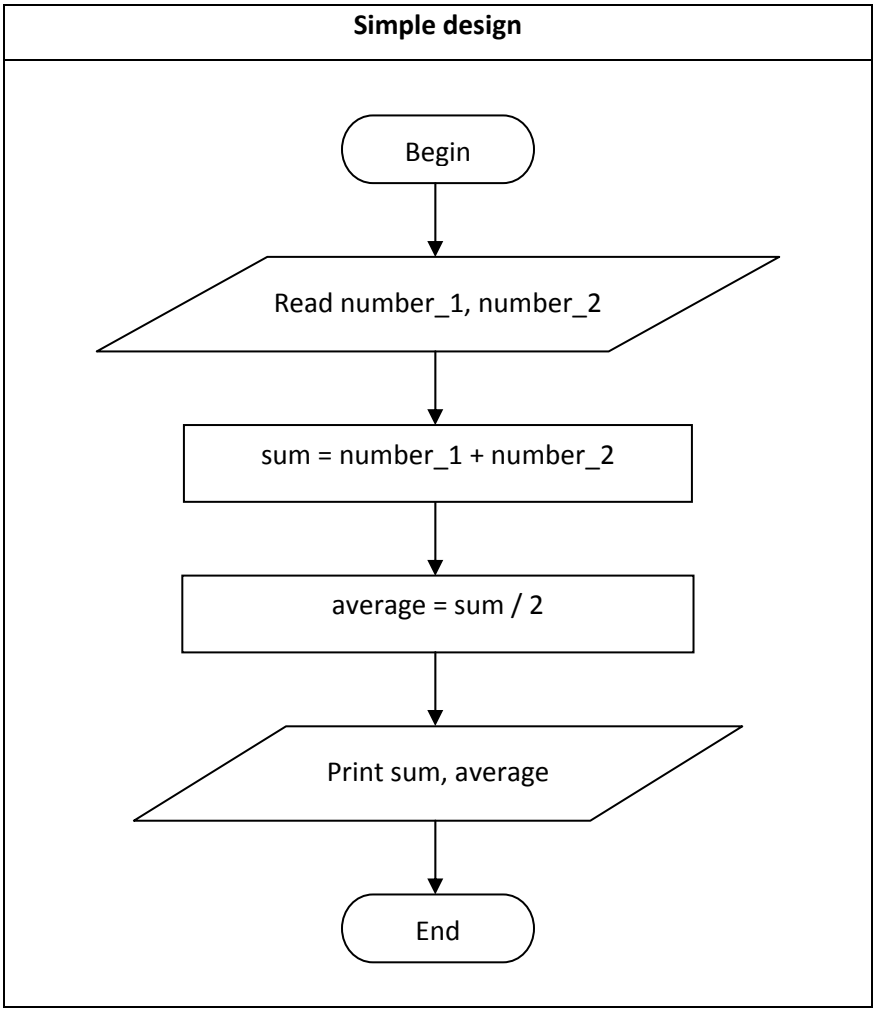
Step 2: Algorithm design (Pseudocode or flowchart)

a. Pseudocode

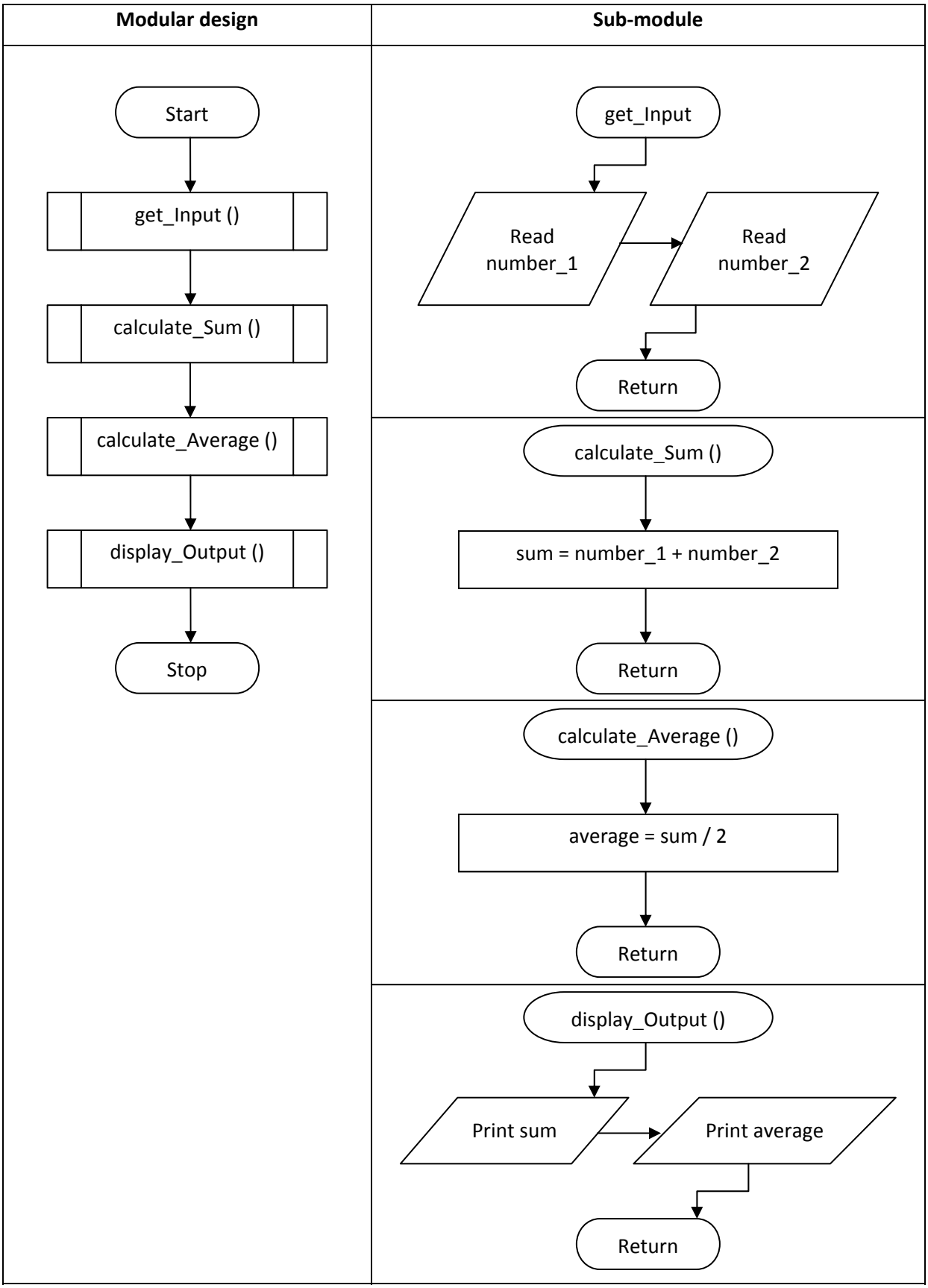
Simple design	Modular design
Calculate the sum and average Begin 1. Get input Read number_1 Read number_2 2. Calculate sum sum = number_1 + number_2 3. Calculate average average = sum / 2 4. Display output Print sum Print average End	Calculate the sum and average Begin 1. get_Input () 2. calculate_Sum () 3. calculate_Average () 4. display_Output () End 1. get_Input () Read number_1 Read number_2 2. calculate_Sum () sum = number_1 + number_2 3. calculate_Average () average = sum / 2 4. display_Output () Print sum Print average

References:
Malik, D.S., (2010). *C++ Programming, From Problem Analysis to Program Design*. Course Technology, Canada.
Gary, R.B., (2010). *C++ For Engineers & Scientists: 3rd Edition.* Course Technology, Canada.

b. Flowchart



References:
Malik, D.S., (2010). *C++ Programming, From Problem Analysis to Program Design*. Course Technology, Canada.
Gary, R.B., (2010). *C++ For Engineers & Scientists: 3rd Edition.* Course Technology, Canada.



References:
Malik, D.S., (2010). *C++ Programming, From Problem Analysis to Program Design*. Course Technology, Canada.
Gary, R.B., (2010). *C++ For Engineers & Scientists: 3rd Edition.* Course Technology, Canada.

Step 3: Algorithm Implementation (Coding)

Simple sequential programming design

```
//Implementation of simple sequential programming design

#include <iostream.h>            //to handle output (cout) and input (cin) statement


int main() // general function
{
      //declare input variable – place a space to store value  in a computer memory
      int number_1, number_2;

      //declare output variable – place a space to store value in a computer memory
      int sum, average;

      cout << "Enter first number ";   //prompt user to enter first number
      cin >> number_1;                 //read value from keyboard and store in number_1
      cout << "Enter second number ";  //prompt user to enter second number
      cin >> number_2;                 //read value from keyboard and store in number_2

      //calculate sum – in CPU and memory
      sum = number_1 + number_2;

      //calculate average – in CPU and memory
      average = sum / 2;

      //Display output (sum and average) to the screen console (monitor)
      cout << "The sum of " << number_1 << " and " << number_2 << " is " << sum << endl;
      cout << "The average of " << number_1 << " and " << number_2 << " is " << average;


return 0;
} //end of main()
```


References:
Malik, D.S., (2010). *C++ Programming, From Problem Analysis to Program Design*. Course Technology, Canada.
Gary, R.B., (2010). *C++ For Engineers & Scientists: 3rd Edition.* Course Technology, Canada.

Simple modular programming design

```
/*
Implementation of simple modular programming design in C++, module also known as function
*/

#include <iostream>        //to handle output (cout) and input (cin)statements

using namespace std;       /*Allow the compiler to use the standard header file
                             defined in ANSI/ISO standard*/

//declare input variable – place a space to store value  in a computer memory
int number_1, number_2;

//declare output variable – place a space to store value in a computer memory
int sum, average;

/*
declare module or function prototype - place a space to store value and processed value in
a computer memory
*/
void get_Input();
void calculate_Sum();
void calculate_Average();
void display_Output();

int main()
{
      get_Input();              //invoke get_Input() function
      calculate_Sum();          //invoke calculate_Sum() function
      calculate_Average();      //invoke calculate_Average() function
      display_Output();         //invoke display_Output () function

return 0;
}

void get_Input()
{
      cout << "Enter first number ";   //prompt user to enter first number
      cin >> number_1;                 //read value from keyboard and store in number_1
      cout << "Enter second number ";  //prompt user to enter second number
      cin >> number_2;                 //read value from keyboard and store in number_2
}

void calculate_Sum()
{
      //calculate sum – in CPU
      sum = number_1 + number_2;
}

void calculate_Average()
{
      //calculate average – in CPU
      average = sum / 2;
}

void display_Output()
{
      //Display output (sum and average) to the screen console (monitor)
      cout << "The sum of " << number_1 << " and " << number_2 << " is " << sum;
      cout << "The average of " << number_1 << " and " << number_2 << " is " << average;
}
```


References:
Malik, D.S., (2010). *C++ Programming, From Problem Analysis to Program Design*. Course Technology, Canada.
Gary, R.B., (2010). *C++ For Engineers & Scientists: 3rd Edition.* Course Technology, Canada.

TUTORIAL

1. What is a computer program?
2. Name **TWO** (2) types of computer languages.
3. What is the only language that a computer can directly understand?
4. Give an example of low-level programming languages.
5. What type of computer language uses English-like abbreviations for machine language instructions?
6. List **FIVE** (5) examples of high level programming languages.
7. What is a compiler?
8. What is the purpose of the preprocessing directive?

```
#include <iostream.h>
```

9. What purpose do braces serve?
10. What are symbols used to write comments?
11. What is a source code?
12. What is an object code?
13. List five basic steps in the programming process.
14. Define an algorithm.
15. Define a flowchart.
16. Define a pseudocode.
17. What is syntax error?
18. What is logic error?
19. What is the purpose of documentation in a program?

References:
Malik, D.S., (2010). *C++ Programming, From Problem Analysis to Program Design*. Course Technology, Canada.
Gary, R.B., (2010). *C++ For Engineers & Scientists: 3rd Edition.* Course Technology, Canada.

PROBLEM SOLVING AND PROGRAMMING ACTIVITIES

1. Write a program to display output as below:

```
*****************************
*         Thank You         *
*    Please come again      *
*****************************
```

2. Write a program to display your name, address and telephone. Print your information line by line.
3. Write a program to display your own message. Your message must at least five lines.
4. People who lives in Shah Alam are imposed a charges 3% income tax on yearly wages. Write a program that will display the income tax.
    a. Identify the output, input and process for the above problem.
    b. Design an algorithm (pseudocode and flowchart) to solve the problem.
    c. Translate the algorithm in C++ programming language.
5. A finance company will give a loan to a customer for buying a car based on the amount of the loan. If the loan is RM 100,000 or more, the interest rate will be 3.5%, otherwise it will be 5% annually. Give the amount of loan and the period of repayment,
    a. Identify the output, input and process.
    b. Draw a flowchart to calculate the amount of interest and the monthly payment or installment.
6. Assume that your weight is 56 kilograms. You need to convert your weight to pound and display it.
    a. Define the output, input and process of this task.
    b. Draw the flowchart.
    c. Write the pseudocode.
    d. Write a C++ program based on the algorithm.
7. The length and width of a rectangle is 5cm and 8 cm respectively. You need to calculate the perimeter and area of rectangle.
    a. Define the output, input and process
    b. Draw the flowchart.
    c. Write the pseudocode.
    d. Write a program based on the algorithm using C++ programming language.

References:
Malik, D.S., (2010). *C++ Programming, From Problem Analysis to Program Design*. Course Technology, Canada.
Gary, R.B., (2010). *C++ For Engineers & Scientists: 3rd Edition.* Course Technology, Canada.

8. Your program should be able to determine whether the applicant is qualified to apply for a credit card or not based on his salary. If the salary is greater than RM1500, then display a message that the applicant entitles to apply. Otherwise, display a message that the applicant cannot apply for the credit card.
    a. Define the output, input and process of this task.
    b. Draw the flowchart.
    c. Write the pseudocode.
9. Your program should be able to determine the correct password. A message, "`Password Accepted`", is display when the user enters the correct password. Otherwise a message "`Incorrect Password, Please Try Again`" will be displayed. The process will repeat until the correct password is entered.
    a. Define the output, input and process.
    b. Draw the flowchart.
    c. Write a pseudocode.
10. Draw a flowchart to calculate and display an average salary of three employees.
11. Draw a flowchart to find the highest salary of three employees.
12. Draw a flowchart to calculate an average of a series of positive integer numbers. The process will stop if the number entered has a negative value.

References:
Malik, D.S., (2010). *C++ Programming, From Problem Analysis to Program Design*. Course Technology, Canada.
Gary, R.B., (2010). *C++ For Engineers & Scientists: 3rd Edition.* Course Technology, Canada.